LITERATURE REVIEW: AN AI-DRIVEN AUTONOMOUS DEVOPS FRAMEWORK FOR INTELLIGENT CLOUD INFRASTRUCTURE PROVISIONING AND APPLICATION DEPLOYMENT

P.Jyothi
Assistant Proffesor, Department of CSE – CyS, DS and AI & DS
Vallurupalli Nageswara Rao Vignana Jyothi Institue of Engineering and Technology
Hyderabad,India
jyothi_p@vnrvjiet.in

G. Vibhas Naidu
Student,Department of CSE - CyS,DS and AI & DS
Vallurupalli Nageswara Rao Vignana Jyothi Institue of Engineering and Technology
Hyderabad,India
vibhasn06@gmail.com

Jagadeeswar Jonnadula
Student,Department of CSE – CyS,DS and AI & DS
Vallurupalli Nageswara Rao Vignana Jyothi Institue of Engineering and Technology
Hyderabad,India
jagadeeswar079@gmail.com

K. Parvez Alam
Student,Department of CSE – CyS,DS and AI & DS
Vallurupalli Nageswara Rao Vignana Jyothi Institue of Engineering and Technology
Hyderabad,India
Parvezz2309@gmail.com

Y. Uday Bhaskar Reddy
Student,Department of CSE - CyS,DS and AI & DS
Vallurupalli Nageswara Rao Vignana Jyothi Institue of Engineering and Technology
Hyderabad,India
udaybhaskarreddy4739@gmail.com

Abstract—The paper provides a literature review concerning the proposed major project, "An AI-Driven Autonomous DevOps Framework for Intelligent Cloud Infrastructure Provisioning and Application Deployment". This review combines 19 base-paper PDFs that deal with AIOps, cloud operations, workload forecasting, AI agents, root-cause analysis, self-healing, Infrastructure-as-Code (IaC), LLM-based Terraform generation, security validation, deployability, and DevSecOps automation. When duplicate versions were screened, 19 separate studies were identified. The literature indicates a development that has taken place from monitoring and anomaly detection to integrated reasoning, agentic decision-making, and autonomous remediation. Nevertheless, the studies surveyed also highlight a continuing gap between code generation and the actual deployment of production-ready infrastructure: the IaC that is generated may be syntactically correct but still not deployable, not in line with user intent, not secure, or not operationally safe. In a similar way, prediction and root-cause analysis are frequently assessed as separate components rather than as part of a closed control loop. The review thus calls for an integrated framework which includes natural-language intent understanding, semantic IaC generation, observability, workload prediction, anomaly detection, LLM-assisted root-cause analysis, policy and security validation, risk-aware remediation, deployment verification, and feedback learning. Keywords: AIOps, Autonomous DevOps, Cloud Infrastructure, Infrastructure as Code, LLM, AI Agents, Self-Healing, Root Cause Analysis, Terraform, Kubernetes, DevSecOps.

I. INTRODUCTION Cloud-native software delivery is now combining microservices, containers, Kubernetes, Infrastructure-as-Code (IaC), CI/CD pipelines and observability. While these technologies enhance repeatability and the speed at which releases can be made, they also generate large amounts of metrics, logs, traces, configuration changes and deployment events. The operational complexity that results makes it difficult to carry out diagnosis and remediation entirely by hand. According to the literature reviewed, AIOps and AI-augmented DevOps are presented as ways of reducing this operational burden while at the same time improving prediction, diagnosis and automation [1]–[4], [11], [15], [17]. Another area of research involves the automation of the infrastructure itself. IaC makes cloud resources declarative and version-controlled, and LLMs are able to convert natural-language requirements into Terraform or Kubernetes configurations [5]–[7], [9], [10], [14], [18], [19]. Nevertheless, the fact that configurations are generated does not ensure correctness. Empirical studies have found discrepancies in terms of functional accuracy, deployment success, security compliance, and user-intent alignment [6], [7], [9], [10]. The project in question thus lies at the point where these two areas meet: autonomous cloud operations and intelligent infrastructure provisioning. The system intended to be developed is not merely a code generator or an AIOps dashboard; it is a closed-loop framework in which an AI component interprets the requirements, sets up the infrastructure, monitors the behaviour at runtime, diagnoses any incidents, suggests or carries out limited remedial action, verifies the result and then passes the outcomes on to future decisions. The studies in question also show that autonomy should be seen as a step-by-step process which is carefully controlled. It is possible to automate monitoring and alerting with relatively little risk, but diagnosis, the generation of configuration settings, and changes to the infrastructure involve ever-increasing operational implications. On this basis, an autonomous DevOps framework should include validation and authorization checkpoints for each action, keep a record of its decisions that can be audited, and provide for a rollback or for human approval whenever confidence in the action is not adequate. From an operational perspective, the literature reviews reveal two types of intelligence that can be useful in the context of cloud operations. First, descriptive and predictive intelligence to transform telemetry into forecasts, anomalies, and incident indicators. Second, generative and agentic intelligence to transform natural language intent, diagnostic evidence, and policy constraints into plans and actions. Both are needed since provisioning an application is not an end but rather the beginning of its effective operation in the cloud.

II. STUDY REVIEW

A. Planning the research

The review was organized around four research questions:

RQ1: What AI/ML and agentic methods have been proposed or applied in the context of cloud operations, incident management, and infrastructure provisioning?

RQ2: How have researchers evaluated IaC generation, deployment readiness, security, and operational reliability?

RQ3: What are the key elements needed to bind observability, predictive analytics, root-cause analysis, decision-making, and remediation into an autonomous reason-loop for DevOps?

RQ4: What are the key research and implementation challenges for a B.Tech-scale autonomous DevOps framework?

B. Conducting the research

The review was conducted using the 20 PDFs provided as part of the current study. Each paper was screened for relevance based on the details provided in the title, abstract, methodology, experiment design, results, and limitations described. One of the provided PDFs (P6) was identified as a duplicate of the semantically enriched IaC study, hence left out from the analysis, resulting in 19 studies included in the review. No unrelated papers were substituted with the ones provided for the initial screening.

The reviewed papers were organized based on the primary contribution of each work, including: (i) AI-driven cloud operations and AIOps, (ii) AI agents and autonomous cloud management, (iii) RCA and self-healing, (iv) LLM/GenAI-based IaC generation, (v) deployability and security validation, and (vi) related architectural or DevSecOps integration. Please note, however, that this categorization is not mutually exclusive. As a result, a given paper could arguably be placed into several of these categories. This review was on the evidence that is related to the project at hand. Architecture, AI techniques, interface, tools, data sets/benchmarks, pertinent metrics, existing limitations, and future directions were considered. Focus was given to the quantitative results shared by the authors. Evaluating approaches: Other than the nine sharing results, the corpus provided insight to the evaluation settings used – such as the use of historic production events, synthetic telemetry, cloud native testbeds, and scenarios from the IaC studies. These differences in evaluation have specific implications that are addressed in some studies; the ability to learn from historic events, to adapt to new environments, and to apply focused validation scenarios for functionality, security or deployability verification. So while a large language model results score provides some insight to the potential effectiveness of a project, it is just one of these factors. In each case, the authors highlighted instances of how they distinguished when an AI recommendation was being given vs. actual cloud operation, tackling the key topic of operational risk. And from our angle for this work, the current project, we need something like that to understand the potential risks of an LLM response - then whether it is enough to move on to the subsequent step, whether it's actual Terraform plan creation, actual Kubernetes manifest deployment, or actual remediation command execution. So the review looked into suggestion vs. execution capability distinctions, the former being recommendations and the latter actually performing recommendations. If quantitative findings were reported, these were evaluated in the context of specific tasks and settings rather than generalized across different areas of application.

C. Thematic synthesis of the reviewed corpus

THEME	PAPERS AND SYNTHESIS
AIOps / cloud operations	[1], [2], [3], [15], [17] — monitoring, forecasting, anomaly detection and operational automation.
AI agents	[4], [11], [12], [14], [15] — agentic interaction, incident lifecycle automation and evaluation.
RCA / self-healing	[3], [4], [8], [12], [15], [16] — diagnosis, explanation, recovery and safety.
LLM / GenAI IaC	[5], [6], [9], [10], [14], [18], [19] — natural-language provisioning and refinement.
Security / policy	[5], [7], [14], [16], [18] — explicit security and compliance validation.
DevOps architecture	[1], [13], [17] — lifecycle integration and architectural coordination.
Benchmarks / evaluation	[4], [10], [12], [16] — reproducible scenarios and standardized metrics.

Table I. Thematic grouping of the 19 supplied PDFs; the categories overlap and therefore are not mutually exclusive.

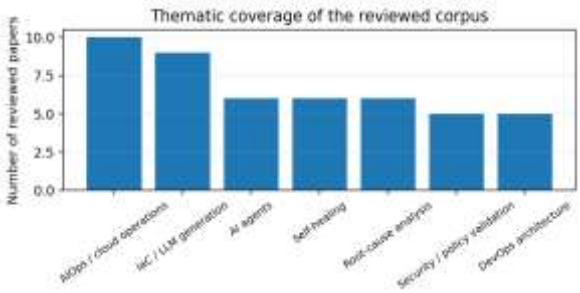


Fig. 1. Thematic coverage of the reviewed corpus (author synthesis; overlapping categories).

III. METHODS AND TECHNIQUES IDENTIFIED

Across the corpus, the literature can be viewed as a pipeline rather than as a collection of isolated algorithms. The first layer is observability and data acquisition: metrics, logs, traces, Kubernetes events, deployment records and configuration context. The seven-layer architecture in [1] explicitly positions observability and contextual enrichment prior to prediction and reasoning. The on-premise AIOps system in [2] demonstrates how Kubernetes telemetry can enable continuous monitoring and workloads forecasting.

Prediction and anomaly detection form the next layer. [2] uses a lightweight Transformer model for workload forecasting, while [17] compares rule-based thresholds, Random Forest and LSTM approaches for cloud telemetry. The evidence suggests that time-series models are useful for proactive resource decisions, but forecasting alone is not an operational decision mechanism. This limitation is explicit in [2], where the authors identify the absence of an integrated decision-making component as a limitation.

RCA and reasoning are addressed by [8], [4], [12] and [15]. RCACopilot aggregates runtime diagnostics and predicts root-cause categories, while AIOpsLab/AgentOps treats incident handling as an end-to-end agent task. The multi-agent framework in [15] separates the workflow into data collection, anomaly detection, RCA and auto-remediation agents. These works support the proposed project's use of specialized modules rather than one monolithic model.

The self-healing literature extends RCA into action selection. The architecture described in [1] separates reasoning, decision-making, safe automation and feedback, while [16] surveys self-healing across the edge–cloud continuum and emphasizes validation, benchmarking, governance and human involvement. Multi-agent solution proposed in [15] distributes duties of data collection, anomaly detection, RCA and remediation agents. While this decomposition can enhance modularity, it also creates coordination overhead and must require an explicit agent-interfaces. A B. Tech implementation would favor a small set of focused modules with a controlling orchestrator over an uncontrolled set of independent agents.

RCA gives context about the causes of the alert. RCACopilot [8] combines diagnostic data with an LLM-friendly pipeline to classify root-cause categories and generate reasoning text. Our evaluation shows Micro-F1 of 0.766 and Macro-F1 of 0.533, with a modest diagnostic overhead of roughly 4.205 seconds. AIOpsLab [12] approaches the problem from an evaluation standpoint by combining workload generation, fault injection, telemetry and agent interaction. These studies then suggest that an autonomous system requires both a reasoning component and a repeatable environment within which the quality of that reasoning can be tested.

Forecasting and anomaly detection have different objectives: forecasting attempts to predict what a workload or resource signal is likely to do next and thus allows for proactive scaling or capacity planning, whereas anomaly detection is concerned with spotting

deviations from normal behaviour. Zhang et al. [2] show the advantages of using Transformer-based workload forecasting in Kubernetes, noting that it achieved an RMSE of 0.11 at a horizon of 24 for AIOpsTrs as against higher errors for the other baseline methods. The study however also points out some limitations, including the short length of the dataset and the absence of integrated decision-making. This supports treating forecasting as an input to a larger control loop rather than treating prediction as an end to automation.

Observability is the foundation of the loop because autonomous decisions rely on current state and historical context. The relevant signals identified across the papers include CPU and memory utilization, request or workload rates, logs, traces, Kubernetes events, configuration metadata, deployment history and dependency relationships. [1] Architectural approach clearly places context, observability, ahead of predicting and reasoning. The envisioned project would normalize these signals in a shared incident context such as the impacted service, recent deployments, resource utilization trends, alerts and other dependencies.

A. Reported quantitative evidence

STUDY / METRIC	REPORTED RESULT
Gajjar [1] RCA	Top-1 70.0 ± 1.5% with dependency/context vs. 31.6 ± 1.6% with anomaly scores alone; unsafe actions 5.7 ± 0.6% vs. 45.8 ± 1.0%.
Zhang et al. [2] CPU forecast	AIOpsTrs RMSE 0.11 versus 0.44–0.58 for the listed baselines.
Yadlapati [17] Anomaly	LSTM: 94% accuracy and 92.5% F1 on the paper's synthetic telemetry evaluation.
Chen et al. [8] RCA	Micro-F1 0.766; Macro-F1 0.533; reported overhead 4.205 s.
Paulo et al. [9] IaC accuracy	64.3% mean functional accuracy versus 28.4% baseline; plan-validation rate improved by 29.6%.
Zhang et al. [10] Deployability	20.8–30.2% first-attempt deployment; iterative framework reported 54.6–91.6% within 10 iterations.
Vargas et al. [7] Security	Syntax and security were not equivalent; one model reached 77.8% validation but 0% Checkov compliance.

Table II. Selected reported results from the base papers. Metrics are not directly comparable because datasets, tasks and experimental conditions differ.

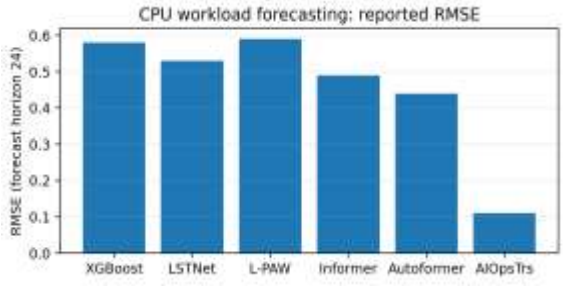


Fig. 2. CPU workload forecasting RMSE at horizon 24 reported by Zhang et al. [2].

The second major category is that of intelligent provisioning. [6] shows that LLMs can be used to generate Terraform from natural language intent, but these outputs are not always deployment ready. [9] tackles ambiguity by using semantic grounding with OWL ontologies that reports a mean functional accuracy of 64.3% compared with 28.4% for its baseline. [10] makes deployability the primary evaluation target: six LLMs initially achieved only 20.8–30.2% deployment success, while iterative refinement resulted in substantially increased deployability across repeated attempts. These findings are directly motivating for a validation-and-refinement loop in the proposed system.

Security is a separate requirement. [7] reports that syntactic validity and security compliance are largely orthogonal and evaluates Terraform with Checkov and Trivy. Therefore, a generated configuration should pass at least four gates before execution: syntax/format validation, policy/compliance validation, security scanning and deployment or plan validation. The DevSecOps copilot vision in [14] further suggests that artifacts from security and other lifecycle stages can be used to refine IaC.

The literature also supports agentic interfaces for cloud management. [11] examines agents communicating through SDKs, CLIs, IaC platforms and web portals while [4] emphasizes standardized environments for evaluating autonomous agents. [12] similarly provides workload generation, fault injection and telemetry interfaces. Together, these works suggest that a project evaluation should include reproducible scenarios instead of a single successful deployment.

The DevSecOps-oriented work in [14] and systematic review in [18] both situate GenAI within the broader lifecycle rather than limiting it to code generation. [14] proposes using artifacts produced at different DevSecOps stages to generate and refine IaC while [18] maps AI techniques across the IaC lifecycle and observes that LLMs are prominent in generation and ML methods are prominent in analysis. The automation architecture described in [13] does just as much as the one based on AI/ML by combining it with model-driven engineering. These results are consistent with a hybrid architecture where generative models reason about intent and explanation while deterministic tools constrain infrastructure and security. Security validation needs to be functionally distinct from syntax and deployability validation. The security-first Text-to-Terraform study [7] tests outputs of Text-to-Terraform against 3rd-party andTerraform validation, Checkov and Trivy, and concludes that several outputs still violate security requirements even when they produce valid syntax. This is a concrete rationale for the design of this architecture: that the various gates should be handled separately: validate syntax and the provider, static security and policy, plan visual validation, test deployability, and test runtime deployability. The success of a property at one stage does not necessarily imply that other properties are fulfilled. Deployability-focused evaluation further supports our claim. Zhang et al. [10] employ a simulation-based workflow, where IaC is executed and, upon error, revised. Their IaC plan achieves a mere 20.8-30.2% first attempt deployment success rate, so generating is not enough; iterated failure-driven redevelopment greatly improves the deployability rate. This fail-learn-refine loop is very relevant to our work because the plan executor's feedback signal, such as the Terraform plan output, Kubernetes validation, execution health, and deployment failures, could be passed to the planning agent as additional signals to guide plan generation.

In terms of infrastructure generation, the literature makes an a priori separation between natural-language interpretation and executable IaC. Mukhopadhyay et al. [6] compare zero-shot, few-shot and fine-tuned LLM architectures with queries rephrased from live Terraform files, noting that the generated plans were not always executable without further editing. Paulo et al. [9] add semantic enrichment with OWL ontologies and report a mean functional accuracy of 64.3% compared to a baseline of 28.4%, coupled with a 29.6% net improvement in Terraform plan validation. This reinforces that domain knowledge and exogenous semantics can diminish the ambiguity inherent in transformations from user intention to platform-specific infrastructure.

IV. RESEARCH GAPS AND PROJECT DIRECTION

• Gap of integration: most works improve one step-forecasting, anomaly detection, RCA, code generation or security-rather than fuse all the steps in a reproducible closed loop [1], [2], [8], [18]. Validation gap: LLM generated IaC can be syntactically correct but not deployable, break intent or have security implications [6], [7], [10].

2 Self-healing approach-gap: an action-safety gap: self-healing research points to actions that are bounded, reversible, and require verification and human intervention before autonomous execution [1], [16].Data Deadlock: autonomous-agent research needs common

benchmarks, real-world failure cases, and long-term data and metrics [4], [12], [16]. Operational feedback deficiency: prediction and generation need feedback on running time, drift checking, and verification after action; otherwise, the system cannot know whether the corrective action restores the desired state [1], [2], [16]. Proposed B. Tech gap: production cloud platforms are too big. A manageable experimental scope: a confined K8/Minikube or K8/Kind testbed with Terraform, Prometheus/Grafana, logs, an LLM planner, validation and a small set of remediation actions. From the literature as well, confidence should be contextual as opposed to encapsulated as a single model score. Data, visualization, and telemetry, dependency graphs, recent deployment history, validation tools and policy checks can be fused to a run-time decision for execution. Safe autonomy can be what is automatically and safely executed such as diagnostics collection, while destructive or high-impact changes necessitate human approval. A further disconnect is between generation quality and closed-loop operational quality. A model may produce semantically valid Terraform but still fail when processing a plan (due to dependency mismatch, resource not available, wrong variable, permission error or runtime condition). Meanwhile, an incident-response agent may successfully identify and issue a normalizing plan but choose an action that triggers a policy violation. The framework should encompass the entire sequence and flag the stage at which failure occurs: intent parsing, plan generation, plan validation, deployment, detection, diagnosis, remediation selection, plan execution, or result verification.

Another example of a missing feature is reproducibility. While a lot of autonomous behavior is driven by the model, it is also driven by the prompts, the context search, the cloud APIs, the provider version, the infrastructure state, and the specific failure mode. A project that saves the prompts, plans, validation, logs of execution, and the final state creates a reproducible trail of your experiments.

A. Proposed integrated framework

The literature describes a project-level architecture which is compatible with the above: The user expresses a natural-language application/infrastructure requirement. The LLM planning layer translates it into structured intent and, if needed, adds to the original request a semantic resource model as [9], then IaC generator produces terraform/k8s artifacts. Before applying the artifacts, syntax, policy and security validation as [7] and plans deployment validation following [10] are performed. Once deployed, observability gathers metrics/logs/traces. Workload prediction and anomaly detection give an early warning; an RCA sub component gathers diagnostic context and generates a ranked explanation (this is the way we follow: [1], [2] and [8]). A decision/agent layer chooses between a small repertoire of safe remediation actions (rollback, restart, scaling, config change).

High-risk actions can require human approval. Execution is followed by verification and feedback, creating a MAPE-K like closed loop consistent with self-healing literature [3], [15], [16].

For evaluation, the project should compare a baseline rule/manual workflow with the proposed AI-assisted loop on controlled failure scenarios. Suggested measures are deployment success rate, time to detect, time to recover, RCA accuracy, false-positive rate, remediation success rate, policy/security pass rate, intent-alignment rate, inference/automation latency and resource overhead. The project should report failures as well as successes; this is especially important because the reviewed literature identifies trust, validation and generalization as unresolved problems.

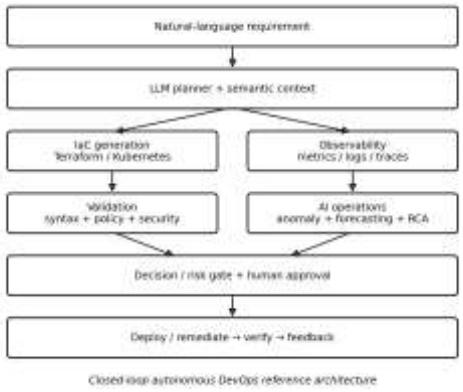


Fig. 3. Proposed closed-loop architecture derived from the reviewed literature.

B. Mapping literature to the proposed system

MODULE	LITERATURE BASIS / PROJECT ROLE
Intent	[9], [11], [18] — convert natural language into structured infrastructure intent.
IaC generation	[5], [6], [14], [19] — generate Terraform/Kubernetes artifacts.
Validation	[7], [10], [14], [16] — syntax, policy, security and deployability gates.
Observability	[1], [2], [12], [17] — collect metrics, logs, traces and events.
Prediction	[1], [2], [17] — forecast load and detect abnormal behavior.
RCA	[4], [8], [12], [15] — diagnose incidents and select actions.
Remediation	[3], [4], [15], [16] — execute bounded fixes, verify recovery and learn.

Table III. Mapping from the reviewed literature to the proposed major-project modules.

Evaluation should report component-level and end-to-end metrics, which can differ substantially. Component metrics could include intent-alignment rate, Terraform validation rate, security-policy pass rate, first-attempt deployment success, RCA accuracy and anomaly-detection performance. End-to-end metrics could include mean time to detect (MTTD), mean time to recover (MTTR), deployment success rate, number of human interventions, rollback frequency and unsafe-action rate. By recording these together, one guards against the possibility that the system merely appears successful by producing more code or automatically executing more steps.

For the proposed B.Tech-scale testbed, Minikube or Kind could be used to provide a reproducible Kubernetes environment, Terraform could represent infrastructure configuration, Prometheus and Grafana could supply metrics, and containerized services could provide controlled workloads and faults. A small LLM-based planner could be connected to deterministic validators and a rule-based execution policy. Fault scenarios could include CPU saturation, pod crashes, unhealthy services, configuration errors and deployment failures. The same scenario should be repeatable for a baseline workflow and for the autonomous workflow so that improvements in recovery time and deployment reliability can be measured fairly.

The execution policy should explicitly define action classes. Read-only actions could include metric queries, log retrieval, It is recommended that cluster owners configure admission controllers to inspect pod specification and deployment configuration during configuration inspection and plan generation. Reversible actions such as restart of failed pod or adjustment of bounded replica count could be allowed. On the other hand, sensitive operations should be managed by policy and require prior approval. Critical operations like resource deletion, exposure change, or persistent storage modifications should be controlled by policy and require prior approval. All actions should be accompanied by the reason,

evidence, command or IaC diff, validation result, authorisation status and the observed state.

A concrete realization can be decomposed into seven cooperating modules. Planning and explanation module, which interprets the user command and synthesizes a concrete deployment goal;Semantic context module, which synthesizes and infers provider, application and policy knowledge,IaC synthesis module, which synthesizes Terra Form and k8s artifacts, validation module, which synthesizes 3 types of checks (syntax, plan, deployability) and 2 policy checks (policy, security);observability module, which syntheses runtime signals, diagnosis and decision module, which correlates a set of alerts to recent changes and dependencies, synthesizes a rootcause, and chooses a bounded action, execution and feedback module, which applies the selected action, validates the resulting system state, and deposits the observed results. Thirdly, the architecture should be constructed such that components can be replaced by other approaches with the evaluation protocol remaining unchanged. For example, other LLMs can be swapped out with the same Terraform, security, Kubernetes, and fault-injection tests. Alternatively, forecasting models can be replaced without significantly changing the overall RCA and remediation interface. This allows incremental construction and comparison of approaches. Supporting this modular design, almost all other literature related to this work is situated as isolated components, such as forecasting, generation or validation, rather than as a single monolith model. Humanl oversight need to be objectivized as a quantified quality attribute, rather than a retrospective fall-back solution. Our approach could realize levels of execution risk, partitioned by action class, resources affected and type of evidence used for validation. Diagnostic actions of low risk could be automate by machine; bounded, safe and reversible actions would need human approval; while more intrusive or externally obvious modifications could be subject to human audit. This log file would provide objective evidence of information gained through self-versal, whether automatised or not. This ability tackles the issues identified in the self-healing and autonomous-agent literature.

The testbed proposed here could be assembled at no additional cost using enterprise cloud platforms. A Kubernetes cluster on which to run a frontend, backend service, database and monitoring stack; for Terraform runs; for visualising the observed metrics using Prometheus and Grafana; and to induce CPU stress, crash pods, cause failed health checks, configuration drift and deployment failures. Detection, diagnosis, planning, verification and act phases would be completed for each case, with the difference being that the dashboard and response would be made to follow the detect-diagnose-plan-verify-act cycle. Repeatedly carrying out such tests would be used to set up a statistical distribution of the result for each scenario.

A closed loop design also changes what is measured for success. A system that produces valid Terraform code but needs manual modification on the same frequency as the existing process is not adding value in the form of code autonomous operation. Similarly, systems that decrease mean-time-to-recover but introduce unsafe manual fixes are not necessarily solving the problem. Instead, such evaluation report should present a very fine grained view of the chain of fulfilment: whether the request parsed successfully, if the IaC code produced was valid, whether it was deployed, whether it had the desired effect, whether the fault was correctly diagnosed, whether the correct root cause was identified, whether the countermeasures were policy compliant, whether the final system was healthy, and other points as shown in [7], [10], [12], and [16]. In this project, context management stands out as a critical design challenge. In general, most cloud operation are stateful, so the same instruction can have a different implication depending on available resources, existing infrastructure, prior changes, and the current state of policy compliance. The semantic encoding as proposed in [9] can also help a trained LLM avoid wrong prescriptive recommendations, it encodes contextual cues into symbolic representations as context vector. Operations research has used state encoding for decades to facilitate planning and scheduling algorithms [1], so in this framework, context can be a hyper-compressed state object embodying the service topology, resource allocations, recent events, relevant logs, policy constraints and consequences of past actions.

From the study the differentiation of probabilistic and deterministic operation is evident. LLMs clearly fit ambiguous operation that require intent recognition, generation of diagnostic hypothesis, verification of policy compliance and cause and effect explanation in natural language. On the other hand, Terraform validation, validation of dynamic-k8s admission control, schema validation, security scan and policy evaluation should be handled by deterministic tools giving trustworthy evidence. The proposed architecture would be a combination of the two, with the LLM handling diagnostics and suggestions, and the validation and execution which would reside as separate binaries and improve the quality of context handling, while supposed transparency and trustworthiness of [10, 11, 14] should therefore avoid following the language model into conclusively determining the result of an execution. Instead, the language model should offer a plan or decision, with other tools assessing the syntax, security, policy and deployability of the request.

Analyzing the studied approaches one by one, each family of methods covers one aspect of the autonomous Dev Ops problem. In AIOps-oriented papers, the aspects of operational visibility, prediction and incident management are tackled, while IaC research studies emphasize the transformation of intent to infrastructure artifacts, agent research work centers on tools usage and decision execution, and self-healing studies on safe recovery. Our framework considers these capabilities as stages of one single lifecycle, which is important as errors can propagate from one stage to another: a faulty interpretation of the intent can lead to erroneous IaC, an insecure configuration can pass the syntax validation, a correct diagnosis can lead to an inappropriate remedial action.

V. CONCLUSION

This review corpus sets a solid starting point for an autonomous Dev Ops project, but reveals the most challenging research problem as being integration and safe operation rather than model accuracy in performing a task in isolation. AIOps research offers observability, prediction and operations intelligence; agent research offers autonomous agency and operator interaction environment; RCA research offers in situ contextual diagnosis; and IaC research offers natural language, semantic grounding, deployability, and security and flaw validation. This project can synthesize these directions into a bounded closed loop for cloud infrastructure provisioning and application deployment. Its contribution should be measured not by offering autonomous production operation, but by demonstrating measurable gains on its reproducible cloud experiment setting, and transparent validation of correctness, security, deployability and remediating safety.

Future work can expand the protoype to more complex dependency graphs, multi-cloud providers, longer operation time-series, more advanced agent benchmarks and adaptive policies. The provided corpus does not determine one globally acceptable architecture and benchmark for all these features; thus, the contribution of this work must be presented as an experimentall-validated integration framework instead of complete cloud autonomy.

The literature's core implication is autonomous Dev Ops is a systems problem. Individually robust techniques-forecasting, LLM-powered RCA, semantic IaC generation, safe code analysis, deployability enhancement, agentic execution-all attack separate failure modes. A promising architecture must orchestrate a systems approach by tying the techniques together through explicit state, validation gates, and feedback. For that reason, this proposal centers on a closed-loop design where the system checks every generated or selected action against evidence of operation and policy prior to execution, then verifies post-execution.

REFERENCES

[1] S. R. Gajjar, "AI-Driven Cloud Operations: An Architectural Framework and Research Challenges," 2026. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6733122.

[2] S. Zhang, D. Fan, Y. Liu, and L. He, "An on-premise cloud-native AIOps infrastructure system with workload forecasting in Kubernetes," SoftwareX, vol. 34, 102622, 2026. doi: 10.1016/j.softx.2026.102622.

[3] V. M. Kancherla, "Autonomous Cloud Operations: The Role of AI-Driven DevOps in Self-Healing Infrastructure," IJERET, vol. 4, no. 1, pp. 66–76, 2023. doi: 10.63282/3050-922X.IJERET-V4I1P108.

[4] M. Shetty et al., "Building AI Agents for Autonomous Clouds: Challenges and Design Principles," Proc. ACM Symposium on Cloud Computing, pp. 99–110, 2024. doi: 10.1145/3698038.3698525.

[5] D. Kumar, "Generative AI for Infrastructure as Code and Cloud-Native DevOps: A Review of Kubernetes Automation, Terraform Generation, Policy Validation, and Configuration Reliability," IJNRD, vol. 11, no. 6, 2026. Available: https://www.ijnrd.org/viewpaperforall.php?paper=IJNRD2606398.

[6] S. Mukhopadhyay, M. Shrivastava, K. Vaidhyanathan, and G. S. Kalahasti, "Leveraging LLMs for Generating Infrastructure as Code: An Exploratory Empirical Study," Proc. ISEC, pp. 1–5, 2026. doi: 10.1145/3796563.3796572.

[7] F. L. S. Vargas, R. B. Mansilha, and D. Kreutz, "Security-First Evaluation of Text-to-Terraform: Benchmarking LLMs and SLMs for Secure IaC Generation," arXiv:2608.02672, 2026. Available: https://arxiv.org/abs/2608.02672.

[8] Y. Chen et al., "Automatic Root Cause Analysis via Large Language Models for Cloud Incidents," Proc. EuroSys, pp. 674–688, 2024. doi: 10.1145/3627703.3629553.

[9] W. Paulo, B. Vasconcelos, and C. Ferraz, "Automating Cloud Infrastructure Provisioning with Semantically-Enriched Large Language Models," Proc. Brazilian Symposium on Multimedia and the Web, pp. 77–85, 2025. doi: 10.5753/webmedia.2025.16102.

[10] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun, "Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation," Proc. ACM FSE, pp. 321–343, 2026. doi: 10.1145/3797142.

[11] Z. Yang et al., "Cloud Infrastructure Management in the Age of AI Agents," ACM SIGOPS Operating Systems Review, vol. 59, no. 1, pp. 1–8, 2025. doi: 10.1145/3759441.3759443.

[12] Y. Chen et al., "AIO PSLAB: A Integrated Framework to Evaluate AI Agents for Enabling Autonomous Clouds," 2025. Available: https://aka.ms/aiopslab-repo.

[13] R. Eramo et al., "An architecture for model-based and intelligent automation in DevOps," Journal of Systems and Software, vol. 217, 112180, 2024. doi: 10.1016/j.jss.2024.112180.

[14] M. Esposito, M. Robredo, A. Bakhtin, D. Taibi, and V. Lenarduzzi, "Generative AI as an infrastructure copilot: automating Infrastructure-As-Code across the DevSecOps lifecycle," Automated Software Engineering, vol. 33, art. 58, 2026. doi: 10.1007/s10515-026-00600-5.

[15] P. K. Nahar and P. Patwal, "Predictive Failure Recovery in Cloud-Native DevOps Pipelines Using Autonomous Multi-Agent AIOps Frameworks," Kuwait Journal of Advanced Computer Technology, vol. 4, no. 1, 2026. Available: https://kuwaitjournals.com/index.php/kjact/article/view/303.

[16] L. Bonilla, J. Diaz-de-Arcaya, J. Aguirre-Usandizaga, and A. Almeida, "AI-driven self-healing across the edge–cloud continuum: A systematic literature review," Information and Software Technology, 108232, 2026. doi: 10.1016/j.infsof.2026.108232.

[17] M. Yadlapati, "AI-Driven DevOps in Cloud-Native Environments: Opportunities, Architectures, and Challenges," International Journal of Intelligent Systems and Applications in Engineering, vol. 14, no. 1s, pp. 521–529, 2026. doi: 10.17762/ijisae.v14i1s.8212.

[18] C. Pahl, Ö. C. Sezen, and F. Hofer, "Artificial Intelligence for Infrastructure-as-Code—A Systematic Literature Review," Electronics, vol. 15, no. 4, 755, 2026. doi: 10.3390/electronics15040755.

[19] D. V. Talati, "AI-Generated code for cloud devOps: Automating infrastructure as code," International Journal of Science and Research Archive, vol. 14, no. 3, pp. 339–345, 2025. doi: 10.30574/ijsra.2025.14.3.0608.